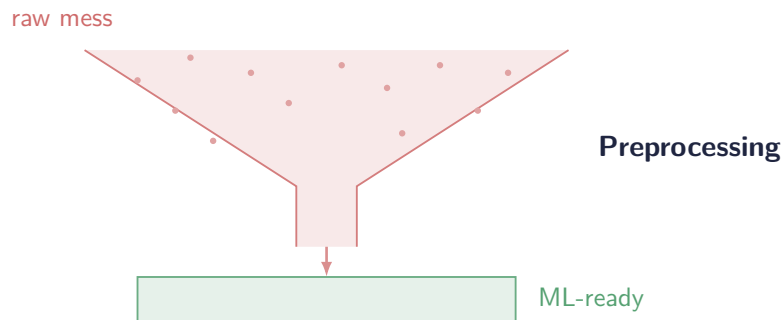


Data Preprocessing Companion

Getting your data ready for a model — told as one connected story.



How to read this. Each chapter answers *three* questions in order: **What** are we doing, **why** are we doing it now, and **how** do we do it. Every chapter ends by pointing at the next one — because preprocessing is one journey, not twelve unrelated topics.

If a name feels strange the first time, keep reading. The picture catches up.

One image to keep in your head: a model is a number-cruncher, and your data is the food you feed it. Every step in this companion is about preparing better food.

Contents

1	Why bother cleaning the data?	3
2	Step 1 — Know your data types	3
3	Step 2 — Look at it (EDA)	4
4	Step 3 — Fill the gaps	5
5	Step 4 — Handle outliers	6
6	Step 5 — Scale columns to a fair race	7
7	Step 6 — Encode categories	7
8	Step 7 — Engineer new features	8
9	Step 8 — Pick the best columns (and squash the rest)	9
10	Step 9 — Handle imbalanced classes	10
11	Step 10 — Honest splits and one Pipeline	11
12	The whole story, in five lines	12
	The preprocessing pocket map	13

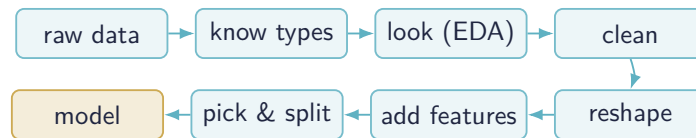
1 | Why bother cleaning the data?

A model is a number-cruncher. It does not understand what your data *means*. If the data is messy, the model just learns the mess. **Garbage in, garbage out.**

That's it. That's the whole reason this document exists.

The journey we are about to take

Eight steps, in order. Each one fixes a specific problem the previous one revealed.



Three honest reasons preprocessing matters.

1. About 80% of an ML project's time goes here. The model itself is the easy part.
2. Every choice — what to drop, what to fill — gets baked into the model. So choose deliberately.
3. A step you didn't write down does not exist. Six months later, your computer remembers; you don't.

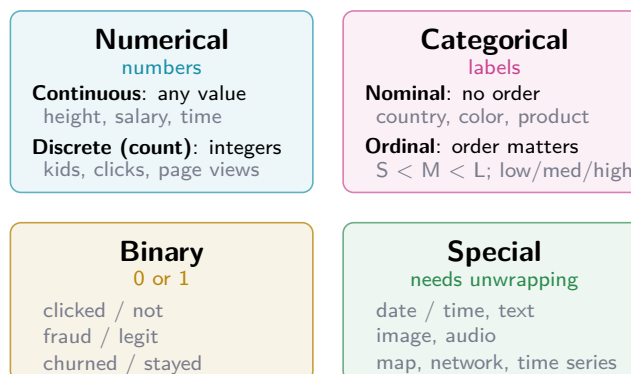
When models in the wild “get better,” it is usually because someone fixed the *data* — not the model. Cleaner data beats fancier algorithms more often than the other way around.

Next: before we touch any column, we have to know *what kind* of column it is. The same trick that helps a number quietly breaks a category.



2 | Step 1 — Know your data types

Every column you will ever meet is one of these. Memorize the four boxes; everything later in the document depends on it.



Why this matters before anything else

Each family has its own toolbox. Mixing them up causes silent damage.

A column called `size` contains S, M, L, XL. You do not know it is ordinal. You fill the missing rows with the “most common” value (M). *Fine*. Then you encode it with one-hot. *Damage*. You just told the model that S, M, L, XL are four unrelated things — and threw away the order that was actually predictive.

Type is destiny. Before any cleaning, fill, scaling, or encoding — write down the type of every column. Five minutes here saves hours later.

Next: we know the types. Now we have to actually *see* the data — shape, gaps, weirdness. That’s EDA.



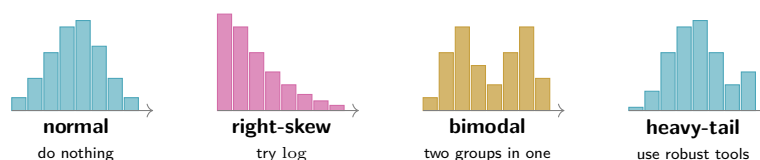
3 | Step 2 — Look at it (EDA)

A friend hands you a box of puzzle pieces. Before you start solving, you tip the box out and look. How many pieces? Any missing? Any from a different puzzle? Five minutes here saves an hour later.

EDA (Exploratory Data Analysis) is just *looking*. You want to find five things, in this order: **shape, distribution, gaps, relationships, weird values**.

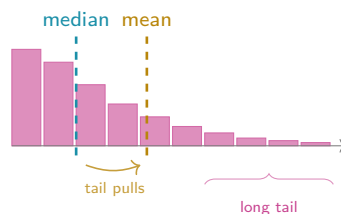
Distribution shapes you will meet

Plot a histogram of every numeric column. You’ll see one of these shapes.



Mean vs. median: who tells the truth?

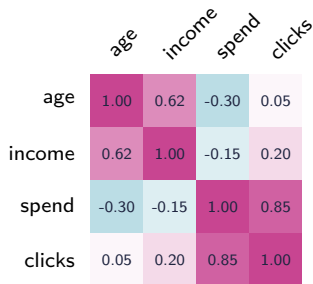
On a symmetric distribution, mean and median sit at the same point. On a skewed one, the long tail *pulls* the mean away — but the median doesn’t budge. That gap is your skew detector.



Quick rule. **mean > median**: skewed right (tail to the right). **mean < median**: skewed left. **mean ≈ median**: symmetric.

The correlation heatmap (relationships in one picture)

Pink: two columns move up together. Cyan: one goes up while the other goes down. Pale: unrelated. Pairs near ± 1 are saying the same thing twice.



What EDA found: some columns are skewed, some have gaps, some have impossible values. The next three chapters fix exactly those three problems.



4 | Step 3 — Fill the gaps

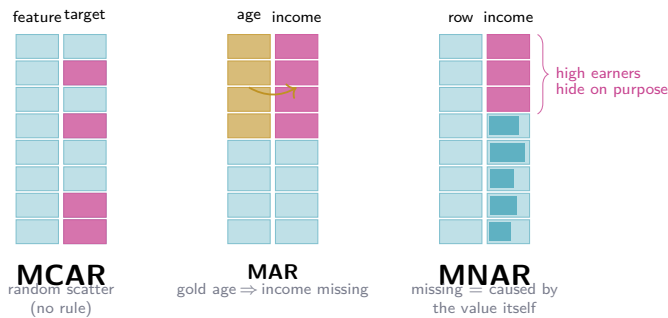
EDA found blank cells. Now we deal with them.

A survey. Some skip the salary question. Were they in a hurry? Did the form glitch? Or did the highest earners skip on purpose? Same blank, very different reasons — and very different fixes.

A blank is data. Always ask *why* it is blank before deciding how to fill it.

The three flavors of missing — spot them by their pattern

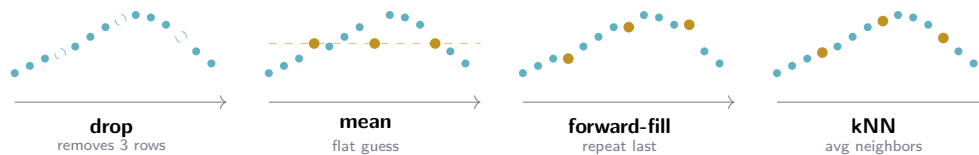
Each picture is a tiny dataset. Rows = people. Two columns: a visible feature (cyan / gold), and a target feature (where pink cells = missing).



MCAR	The form glitched. Drop rows or fill simply. Easiest case.
MAR	Missing depends on a column you can see. Fill using the other columns (kNN, model).
MNAR	Missing depends on the missing value itself. Fill carefully <i>and</i> add a “was missing” flag. Hardest case.

Filling a gap, four ways — on the same series

The hollow rings below were missing. Each panel uses a different rule to guess them.



Compute fill values from the *training set only*. If you compute the mean over the whole dataset before splitting, the test data secretly leaks into your model.

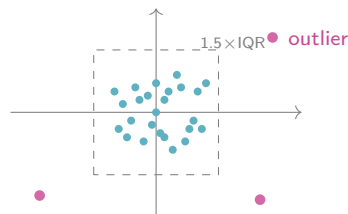
Even when you fill, add a 0/1 flag column saying “this row was originally empty.” That fact alone often predicts the target.

Next: EDA also flagged some points sitting far from the rest. Are they typos, real-but-rare events, or the very thing we want to catch?



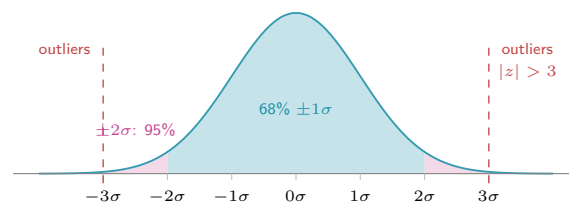
5 | Step 4 — Handle outliers

A teacher reviews exam scores. Most are 40–90. One is 350. Typo? Or a cheater who copied 350 questions? You cannot decide by the number alone.



The z-score rule, drawn

A z-score says “how many standard deviations is this point from the mean?” Most data sits within $\pm 2\sigma$. Anything past $\pm 3\sigma$ is genuinely far away — about 0.3% of points in a normal distribution.



Three steps, in order

- Find them.** z-score > 3 , or outside the box-plot whiskers ($1.5 \times \text{IQR}$), or use Isolation Forest.
- Look at them.** Plot them. Read the rows. Ask the domain. *Never* delete blindly.
- Decide.** Cap, transform with log, keep with a robust model, or remove only if proven an error.

In fraud detection the outliers *are* the answer. Delete them and you delete the very thing you are trying to find. Always ask: noise, or signal?

Next: the data is clean, but the columns live on wildly different scales. Salary is in the millions; age is in single digits. Most models will pay attention only to salary unless we fix that.



6 | Step 5 — Scale columns to a fair race

A track meet where one runner gets a 100-meter head start. Of course they win. That is what happens when one column is in the millions and another is in single digits.

Models that use distance (k-NN, k-means, SVM, neural nets) get dominated by whichever column has the biggest range. Scaling moves every column to a fair starting line.



Pick by what your data looks like

Method	What it does	Use when
Min-Max	Squashes into [0, 1]	Bounded output needed (e.g. neural nets)
Standard (z-score)	Centers at 0, spread 1	Default for linear models
Robust	Uses median and IQR	You have outliers
log / log(1 + x)	Squashes long right tails	Income, prices, counts
Quantile (rank)	Replaces value by its rank	Any monstrous shape

Income: [1,000, 50,000, 80,000, 200,000, 5,000,000].
After log: [6.9, 10.8, 11.3, 12.2, 15.4]. The 5,000× gap shrinks to 2×.

Compute scaling values on the *training set only*. Apply the same numbers to train, validation, and test.

Next: the numerical columns are scaled. But the categorical columns aren’t even numbers yet. Time to translate.



7 | Step 6 — Encode categories

A model can only do math. “Red” / “Blue” / “Green” must become numbers — without lying about order.

You label Red = 1, Blue = 2, Green = 3. The model thinks Green is bigger than Blue and “three times” Red. But the colors have no order. You taught the model a lie.

column		one-hot	label	target					
Red		<table><tr><td>1</td><td>0</td><td>0</td></tr></table>	1	0	0	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0.18</td></tr></table>	0.18
1	0	0							
0									
0.18									
Blue	→	<table><tr><td>0</td><td>1</td><td>0</td></tr></table>	0	1	0	<table><tr><td>1</td></tr></table>	1	<table><tr><td>0.42</td></tr></table>	0.42
0	1	0							
1									
0.42									
Green		<table><tr><td>0</td><td>0</td><td>1</td></tr></table>	0	0	1	<table><tr><td>2</td></tr></table>	2	<table><tr><td>0.61</td></tr></table>	0.61
0	0	1							
2									
0.61									

Pick by cardinality (how many unique values)

Few (< 10) → one-hot. **Some** (10–100) → one-hot if model handles width, else target.

Many (100s–1000s) → target encoding (with cross-validation), or frequency.

Huge (millions) → feature hashing or embeddings.

Ordinal values (S, M, L) genuinely have an order. Use ordinal encoding for those, *not* one-hot. The order is the predictive signal — don’t throw it away.

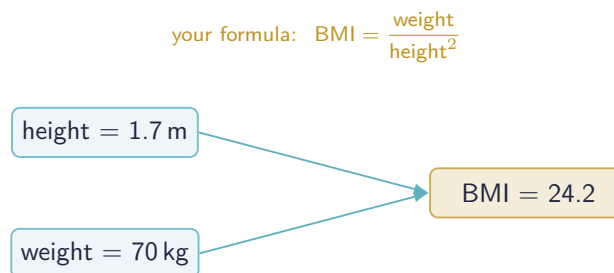
Next: every column is now numeric and on a fair scale. But the model can only see the columns we gave it. It will not invent BMI from height and weight by itself — so we hand it BMI.

◇

8 | Step 7 — Engineer new features

You have height and weight. The model sees two numbers. A doctor would compute BMI = weight ÷ height². You hand the model a number doctors actually use — before it ever has to discover the idea.

In classical ML, this is the single highest-impact thing you can do. A clever new column beats a fancier model nearly every time.



Patterns that almost always help

Aggregations: avg / count / max of a user’s recent activity.

Time differences: days since signup, hours since last click.

Ratios: price/income, clicks/views, debt/income.

Date parts: day of week, hour, season, holiday flag.

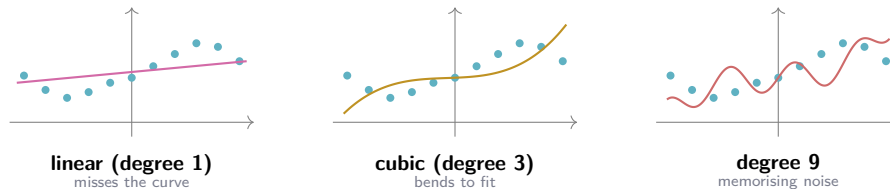
Past values (time series): yesterday’s value, rolling 7-day average.

Expert formulas: BMI, PE ratio, APR, etc. *This is the gold.*

A real-world swap. A churn-prediction team replaced `plan_id` with `days_since_last_login`. Same model. Score went up 4%. *One* good feature beat a week of model tuning.

A line cannot draw a curve — unless you give it x^2

A linear model fits a straight line. Feed it x^2 as an extra column and that “straight line” starts to bend. Feed x^3 too and it bends more. This trick is called *polynomial features*.



More degree is not better. Beyond a point, the curve stops fitting the truth and starts memorising the noise — that is overfitting. Keep degrees low (2 or 3 usually) unless you know you have lots of data.

Next: we just added a bunch of columns. Some of them won't earn their keep. Drop the dead weight before training.



9 | Step 8 — Pick the best columns (and squash the rest)

Packing for a trip. Pack everything you own and you struggle at the airport. Pack what you need and the trip is easier. Selection is packing for the model.

Three families of selectors

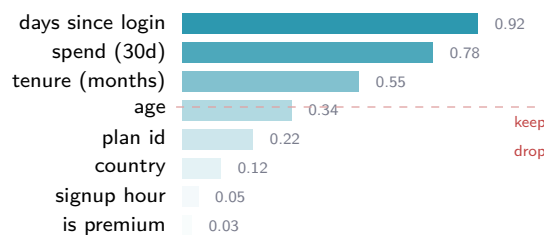
Filter → score each column on its own. Drop low-variance and low-relevance. Cheap.

Wrapper → try sets of columns, train a real model, pick the best set. Expensive.

Embedded → let the model itself rank columns (Lasso, tree importance). Free.

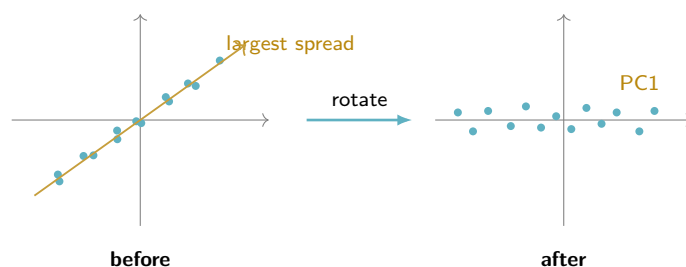
What “feature importance” looks like

After training a tree-based model, every column gets an importance score. Sort, plot, keep the top few.



When you cannot drop, compress: PCA

PCA finds the rotation that puts the most spread on axis 1, the next most on axis 2, and so on. Keep the top axes; drop the rest.



If PC1 captures 95% of the spread, you just turned 100 columns into 1 with almost no loss.

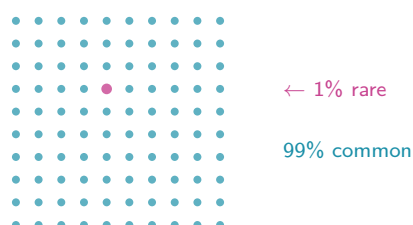
Next: our column set is lean. But what if the target itself is lopsided — 99% one class, 1% the other?

◇

10 | Step 9 — Handle imbalanced classes

A doctor sees 100 patients. Only 1 has a rare disease. If she always says “you are fine,” she is right 99 times — and misses the one patient who needed her. The model has the same temptation.

99% accurate, 0% useful.



Six tools, in order of preference

Use better metrics first. F1, precision, recall, PR-AUC. Stop using accuracy.

Tune the threshold. Move the decision cutoff to catch more rare cases.

Weight the loss. Tell the model that missing a rare case hurts more.

Oversample the rare class. SMOTE invents new synthetic minority points.

Undersample the common class. Drop some majority rows.

Reframe as anomaly detection when the rare class is below 1%.

SMOTE: invent new rare points



Pick a real minority point. Pick its nearest minority neighbor. Drop a fake point on the line between them. Repeat. You now have more rare-class examples without literal copies.

SMOTE *after* splitting train/test, on the training data only. Otherwise synthetic points leak into your test set.

Final step: we are about to train a model. But every transformation we just learned — fill rules, scaling values, encodings — has to be applied honestly. One pipeline, one contract.

◇

11 | Step 10 — Honest splits and one Pipeline

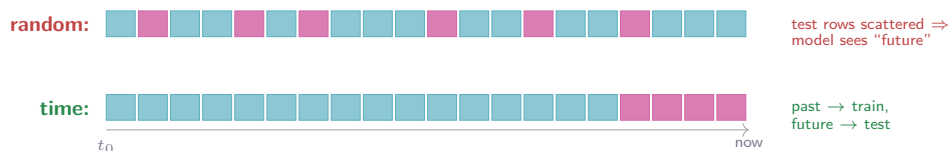
A student studies last year's exam, then takes last year's exam as the real test. Of course they ace it. The score tells you nothing. A bad train/test split is the same trick, played on yourself.

Four ways to split

Random (i.i.d. rows) · **stratified** (keep class ratios) · **by group** (no user in both sides) · **by time** (train past, test future — mandatory for time series).

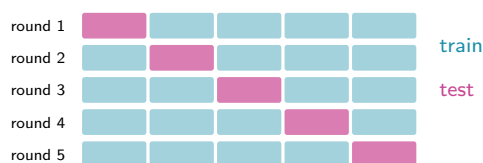
Random vs. time split, side by side

For time series, a random split lets the model see the future. A time-based split keeps the contract honest.



Cross-validation in one picture

Split into k pieces. Train on $k-1$, test on the last. Rotate. Average. Below: $k = 5$.

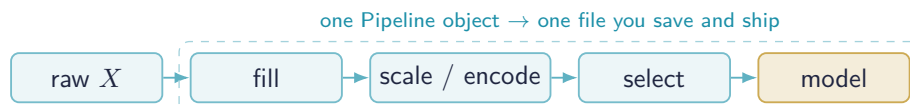


The four kinds of leakage to fear

- (1) **Target leakage** — a column that won't exist at prediction time (e.g. `paid_at` when predicting `will_pay`).
- (2) **Train mixed with test** — scaling values, fills, encodings computed on the whole dataset.
- (3) **Same person in both sides** — same user in train and test for a personalisation task.
- (4) **Future in the past** — rolling stats computed using future values.

The Pipeline: the artifact you ship

Every step in this whole document lives inside one Pipeline object. You save the Pipeline, not the model.



Golden rule. Anything you fit on the data — a scaler, a fill rule, an encoder, a selector, PCA — must see *only* the training data. Then you transform train, validation, and test with the same fit. The Pipeline does this for you.

◇

12 | The whole story, in five lines

1. **The data is the model.** About 80% of ML is data work. Garbage in, garbage out.
2. **Type is destiny.** Always know what kind of column you are touching.
3. **Look before you fit.** Five minutes of EDA saves hours later.
4. **Clean, reshape, engineer, select — in that order.** Each step prepares the next.
5. **The Pipeline is the contract.** Fit on train, transform train and test the same way. Ship the whole pipeline.

One image to keep: a model is a number-cruncher and your data is its food. Every step of preprocessing is about preparing better food.

One question to keep asking: *would this column actually exist at prediction time?* If no, drop it. That single question stops most data leakage in its tracks.

The preprocessing pocket map

A printable one-page recap of the whole journey.

1 Know your types



Numerical (continuous, count) · Categorical (nominal, ordinal) · Binary · Special.

Type decides every tool you use.

2 Look before you fit



Shape, distribution, gaps, relationships, weird values.

Five minutes here saves an hour later.

3 Fill the gaps



Ask *why* it's missing first. Drop / mean / kNN / model. Always add a "was missing" flag.

The blank itself is signal.

4 Strangers: noise or signal?



Find ($z > 3$, IQR), *look* (never delete blindly), then decide: cap, log, or robust model.

In fraud, the outliers are the answer.

5 Fair race



Standard, robust, min-max, log. So one column doesn't drown the others by sheer size.

Fit on train only. Apply to all.

6 Words → numbers



Pick by cardinality: few → one-hot, many → target, huge → hashing or embeddings.

Never imply a false order.

7 Add what the model can't see



BMI, ratios, dates, time-since, rolling stats, expert formulas. Domain knowledge = a free head start.

One smart feature beats a fancier model.

8 Pack light



Drop low-variance / low-relevance. Use Lasso or tree importance. PCA when you can't drop, only compress.

More columns is not better.

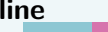
9 Don't reward laziness



99% accuracy on imbalanced = useless. Use F1, tune threshold, weight loss, SMOTE.

Stop using accuracy.

10 Honest split, one Pipeline



Random / stratified / by group / by time. Fit *everything* on train only. Ship one Pipeline file.

The pipeline is the artifact.

*Garbage in → garbage out. Garbage in → **cleaned** → a model that earns its keep.*

End of companion. Now go fix some data.